

I should **first acknowledge** that I'm basically being carried by my teammate here. What's been completed **is all his work**. I have some **ideas but unfortunately** I didn't have time to finish them. But I'll **integrate** those ideas to the presentation.

Chassis

So um, I made another iteration of the chassis in **FreeCAD**. It's meant to be more **modular** and **lighter** plus some **minor improvements**. However the biggest **problem** I created is that it's hard to **assemble** because the **screws** are hard to access. I plan to change to **horizontal screw tabs** next time.

Signal Processing: IMU

The IMU has gyroscopes and accelerometer, and in some chip packages like mine, has a magnetometer as well. The **gyroscopes measures** the angular velocity and the **accelerometer measures** the acceleration.

To get the pitch we need to do **sensor fusion**. It can be done by either **FOSS libraries or the proprietary DMP** on the MPU6050.

The reasons are:

1. None of the sensors measure positions, only **derivatives**. If you just integrate gyro without an absolute reference you only have a **relative position** to where it started. The accelerometer gives a **vertical reference** which is the gravity, for the pitch and roll. And the magnetometer gives a **horizontal reference** which is north, for the yaw.
2. Sensor fusion algorithms **compensate for the weakness** in each sensor that can't be removed by **calibration**. (e.g. temperature drift in gyro.)

I use the cheapest and fastest algo called **Mahony**.

Internally those algorithms use **Quaternions** to represent orientation, which is better than Yaw Pitch Roll or Euler angles, because it doesn't have the **gimbal lock problem** (You lose a DoF and can't move in that direction in some orientation). But for our use case, **pitch is the most straightforward**.

Signal Processing: PID controller

The **angle** can deviate from the balanced state. The **PID controller** determines how much to **drive the motor** to fix the error.

There are three parts of PID controller. The **explanation in the instruction** is pretty good, but here's a breakdown:

1. P - Proportional. The proportional response is **proportional to the error**. (Error is how much the angle deviates from neutral.)
2. I - Integral, looks at the *past*. The integral response is based on the **accumulated error** over time. It drives the **steady-state error to 0**. Because for as long as an **error exists, it continues to grow**.
3. D - Derivate, predicts the **future**. It looks at the **trend** of the error and adjust accordingly.

It's very important to find a good combination of these coefficients. But here is a trick, we can just use the **gyro reading for the derivative term**. The Y angular velocity is exactly the change in pitch. You can do this in the **AdvancedPID** library,

The calibration steps:

1. Start with every thing at zero.
2. Tuned the set point (balanced angle) and K_p together, until the robot oscillates in both direction with similar probability.
3. Use gyro Y for K_d .
4. Increase the K_i , so that the steady-state error is 0 (does not keep moving in one direction without restoring the angle).
5. Trial & Error fine tuning.

Control Logic

The control loop is basically shown like here.

We read the pitch and gyro, checks what to do, calculates PID output, then drive the motor.

My difference was that I use libraries for every component, so that it's more straightforward. **Separating** myself from **calculations** and **raw input / output**.